Learned Layers for a Factor-Based PDE Pipeline: What Worked, What Did Not, and Why

Flavio Pricoco*

November 7, 2026

Abstract. A pipeline that reads the factors of a linear partial differential equation from noisy data already contains classical statistical learning: a Gaussian process interpolates, a Dirichlet-process mixture groups, a Bayesian linear model splits components. This paper asks a different question: can *learned* layers, trained on simulated equations, make that pipeline more accurate, better calibrated or faster, without ever changing what it is allowed to claim? We state a contract under which a learned layer may propose a candidate, fit a global constant, or raise a flag, but never decide a label, and under which every layer switched off reproduces the pipeline bit for bit. We designed eight such layers, built seven, and gated each on held-out equation families, with leave-one-family-out validation, paired cluster bootstrap and conformal operating points. One layer ships: an exact memo that is bit-identical to the pipeline and saves about half a percent of the time. A learned failure detector separates injected failures from clean runs (cross-validated AUROC 0.85, against 0.63 for the pipeline's own omnibus test) but has almost no detection power at the 5% false-alarm level it needs, and a controller that reruns the pipeline with other settings has an oracle ceiling of 8 of 12 failures, reduced to 0 of 12 once a real trigger is required. The calibration layers fail because the quantity they would calibrate is saturated or already calibrated. We report each negative result with its numbers, the lessons they share, and the ideas that remain.

Key words. equation discovery, uncertainty calibration, conformal prediction, failure detection, negative results

MSC codes. 35E20, 62F15, 62G15, 68T05

1 Introduction

The pipeline studied here takes noisy observations of a field $u(t, x)$ that solves a linear constant-coefficient equation and returns a distribution over equations, each with intervals on its coefficients and one of four labels: FIT-COMPATIBLE, MISFIT, ID-KER, UNDECIDED. It never prints that a law is identified. Its design and the mathematics behind it are in the companion papers; here we only need its behaviour. On a development benchmark it is right on all thirty inputs and never wrong without warning. On twenty random equations it returns the right or an equivalent equation on nineteen. Yet on a larger simulated corpus of random equations from five families, on the earlier pipeline version the layers were built on and with every learned layer off, it is *false-confident* on 9 of 80 held-out grid cases: it reports FIT-COMPATIBLE for a wrong factor set.

That figure is the motivation. Is there a learned component, trained on simulated equations, that removes such errors, or that tells us when they occur? The question matters beyond this

*flaflapr@gmail.com

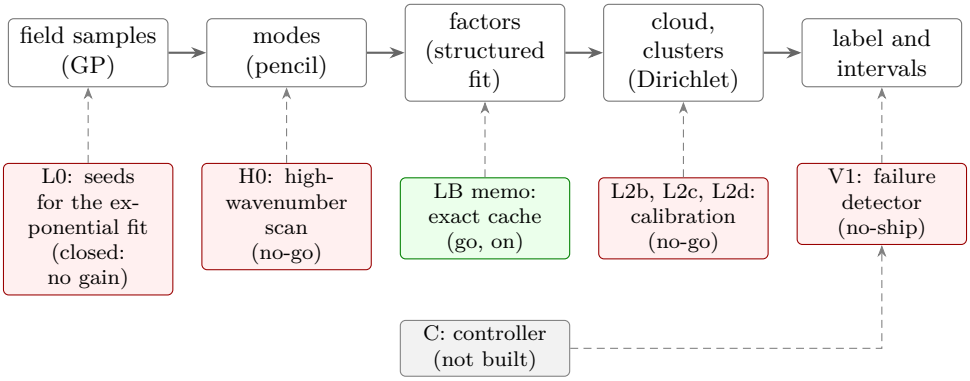


Figure 1: The pipeline (top) and the learned layers attached to it (below). Green: shipped; red: gated and rejected; grey: designed, not built. Dashed arrows show the step each layer touches. V1 only reports; C would have used V1 as its trigger.

pipeline, because learned correction layers are the usual answer to residual miscalibration in scientific methods, and rarely come with a pre-registered gate and a report of the layers that failed it.

What we did We defined a contract for learned layers (Section 3), built the layers the pipeline admits (Section 2), and evaluated each against a gate fixed before the run, on corpora of simulated equations that do not overlap the development benchmark (Section 4).

What we found Of eight layers (seven built), one is on by default and it is a pure speed-up (Section 5). A learned failure detector is the only layer with a positive effect on the quantity it targets, and that effect does not survive the operating point the contract requires (Section 6). The others are negative: a learned skip of optional steps saves at most 0.7% of the time (Section 7.1); calibration of interval widths and of the top-structure probability does not help (Section 7.2); a scan for high wavenumbers never changes an answer (Section 7.3); a controller that chooses reruns has an upper bound that no available trigger reaches (Section 7.4). Table 1 lists all verdicts. Section 8 extracts what the failures have in common, and Section 9 lists ideas not yet tried, including the author's own.

Scope Everything here concerns linear, constant-coefficient, autonomous, scalar equations in one space and one time variable. The layers were developed on one version of the pipeline (Section 10); we say so wherever it matters.

2 The pipeline and where learning can enter

Figure 1 shows the pipeline as a chain of five steps and the places where a learned layer can attach.

Learning already inside the pipeline Three components are statistical learning and are part of the pipeline, not optional layers: a Gaussian process (with scalable variants) that interpolates scattered data into samples of the field [4]; a Dirichlet-process mixture over the

factors of the posterior samples; and a Bayesian linear model that splits the field into one component per factor. They are described and benchmarked in the companion papers. They are fixed here.

The layers Optional layers attach at the places in Fig. 1. L0 proposes starting values for the exponential fit that extracts modes. H0 scans the raw residual for wavenumbers the Gaussian-process path misses. LB caches repeated mode extractions (exact) or learns to skip optional steps. L2b, L2c and L2d calibrate, respectively, the probability of the top structure, the interval widths, and the global constants of the decision rules. V1 predicts from the pipeline's own output whether the answer is wrong. C would read V1 and rerun the pipeline with other settings.

3 The contract for a learned layer

1. **Off is today, bit for bit.** With every layer off the output equals the pipeline's output exactly: factor strings, labels, intervals, evidence. This is a test, on stored outputs, not a promise.
2. **Propose, calibrate or flag; never decide.** A layer may output a candidate (a mode, a structure) that the pipeline's own evidence then scores; a global constant or scale fitted offline; or a report flag. Labels come only from the deterministic evidence and the identifiability rules.
3. **Fixed likelihood.** Nothing learned changes the noise model or the Occam penalty for a run.
4. **Monte Carlo, not linearisation.** Interval scalings act on the Monte Carlo draws themselves.
5. **A gate before switch-on.** A layer is switched on only if it passes held-out evaluation on equation families it was not trained on (leave-one-family-out), with calibration at least as good as the pipeline's and clean null runs. A layer that fails stays off; its code, its tests and the measured reason stay in the repository.
6. **Scale-free inputs.** Learned features are dimensionless, so that a rescaling of the coordinates does not change a decision.

Rules 1 and 2 are what make a negative result cheap to obtain and safe to keep: a failed layer is switched off and the pipeline is unchanged.

4 Protocol

Corpora Equations are drawn from five simulator families F1 to F5 of random factored operators with exact dispersion relations. Corpus G has grid inputs at 1% and 5% noise; S has scattered inputs; N has nulls (pure noise and single-factor equations); V has 100 clean 5%-noise grid runs (fit seeds 600–607, calibration 608–615, test 616–619) that were added so that a conformal gate at level 0.05 can be evaluated; and I has injected failures.

Injected failures Corpus I has seven types at three strengths: two laws in two time halves, a weak nonlinearity, a variable coefficient, a fractional symbol, coloured noise, heteroscedastic noise, and aliasing. Strengths are scale-free: for deviations of the law, $z =$

$\|u_\varepsilon - u_0\|^2/(\sigma^2\sqrt{2N}) \in \{2, 8, 32\}$; for coloured noise, an AR(1) coefficient in $\{0.3, 0.6, 0.9\}$. Aliasing is a frame shift, $\tau \mapsto \tau + V\xi$, which keeps the factor structure and pushes frequencies beyond the Nyquist limit; a time rescaling would have moved the draw out of the simulator's prior.

Statistics Effects are paired differences with a 95% cluster bootstrap over base draws (seeds), because many injected rows are variants of one draw and a row bootstrap would be too narrow. Validation is leave-one-family-out (LOFO) and, for injection types, leave-one-type-out. Operating points use split or cross-conformal calibration [2] and conformal risk control [1]. Learned models are gradient-boosted trees [3] or logistic regression on dimensionless features.

Gates A layer ships only if its gate passes in every LOFO fold, not on average. We report "go", "no-go" or "unclear", the last when the sample cannot decide it.

5 What worked: an exact speed-up

The mode extraction and the confluence test inside the core depend only on the sample, the noise and the mode options, not on the structure being tried. Within one run the pipeline calls the core repeatedly on the same sample (growth, refinement, re-runs), so a memo keyed on these inputs returns identical results. Profiling found the confluence test at 80–90% of core time.

The memo is exact by construction and by test: 0 of 47 paired held-out runs, 10 null pairs, 10 named cases and 10 timing pairs differ in any label, factor string, coefficient or weight. On the pipeline version of the first measurement the total runtime ratio was 0.928 [0.896, 0.948]. After the pipeline's own later speed-ups (a faster confluence test) the same measurement gives 0.995 [0.992, 1.000]: the memo is now worth about half a percent. It stays on because it is exact and harmless, and because its cost is a few lines. This is the only layer that ships, and its lesson is also a warning: a speed-up measured against a slow baseline can disappear when the baseline improves.

6 The failure detector V1

V1 is a gradient-boosted classifier that takes dimensionless features of the pipeline's output (labels, evidence margins, residual statistics) and predicts whether the answer is wrong. It is report-only: it changes no label. A variant would downgrade a flagged FIT-COMPATIBLE to UNDECIDED.

Feasibility On the first pipeline version, over 49 clean and 92 scored injected runs, LOFO AUROC is 0.900 [0.825, 0.975] against 0.681 for the pipeline's own omnibus residual test (difference +0.219 [+0.078, +0.289]); on FIT-COMPATIBLE runs only, where an undetected failure matters most, 0.759 against 0.515, which is chance. On fresh base draws the estimate is about 0.85, because one seed supplies 21 of the 28 injected rows per family and those rows score higher.

Re-gate on the final pipeline With 100 clean 5% grid runs and corpus I re-run, LOFO AUROC is 0.847 [0.720, 0.914] and exceeds the omnibus test by 0.222 [0.018, 0.356]. But the conformal operating point at $\alpha = 0.05$ fails: the false-alarm rate is inside the bound (split 0/17, LOFO 0.00–0.09), and detection is 0.00–0.08 per fold, below the omnibus test's 0.26–0.47. The score separates on average but not in the far tail that a 5% false-alarm level demands. The downgrade variant, replayed on held-out rows, leaves false-confident answers at $7 \to 7$ on clean rows (losing two correct answers) and $3 \to 2$ on injected ones. **Verdict: no-ship.**

Why V1 reads the pipeline's output, so it cannot flag a run that crashes or times out (48 of 140 injected runs on the first pipeline version); it detects none of the injected nonlinear, variable-coefficient or two-law failures on the final pipeline; and an output-only detector inherits the blind spots of the pipeline that produced the output.

7 Layers that did not help

7.1 Learned skipping of optional steps (LB)

Optional steps (grow, recurse, complex, alternatives) can be skipped when a classifier predicts that they leave the state unchanged. A perfect classifier would save 17% of total time on the fitting and calibration runs (14% on test). The zero-harm calibration, cross-fitted over 165 runs, saves 0.2% (harmed runs 0/165); a 2%-risk profile saves 0.7% with 2/165 harmed. On the test split neither skips anything. The largest guarded step changes the state in 64 of 133 runs, and most time is spent in steps that change the state every time and are not optional.

7.2 Calibration: P(top), interval widths, constants

L2b, probability of the top structure The raw probability is at least 0.999 on 96% of rows, so it has almost no resolution; a calibrator that sees only it learns a base rate, and base rates differ by family (right-or-equivalent: 1.00 on F1, 0.77 on F2, 0.82 on F3, 0.36 on F4, 0.75 on F5). Best Brier change for right-or-equivalent is -0.032 [-0.073, $+0.007$], and every arm is worse than raw on F1 where raw is already perfect. *No-go.*

L2c, interval widths An earlier experiment on grids with a stand-in posterior moved coefficient coverage from 0.756 to 0.923 on held-out families. On the final pipeline and 114 test cells the pipeline's own 90% intervals cover 0.895; a scalar rescaling model gives 0.965 and a gradient-boosted model 0.982, wider intervals that over-cover, and neither lies in $[0.85, 0.95]$ on every fold. The earlier gain was a repair of a defect the pipeline had since fixed. *No-go.*

L2d, global constants Tuning the evidence thresholds and the minimum signal-to-noise by Bayesian optimisation chooses today's values in every LOFO fold (change +0.000 [0, 0] for the signal-to-noise cut). *No-go.*

7.3 High-wavenumber scan (H0)

A raw-data matched filter followed by variable projection reaches the Cramér–Rao bound at wavenumber $\kappa \in [1, 4]$ on heat data (400 points, 5%) and recovers $\kappa = 4$ that the Gaussian-process path misses; beyond $\kappa = 5$ the amplitude is below 0.4σ and no method can. Inside the

Table 1: Verdicts on all layers. "Gate" is the pre-set criterion; numbers in brackets are 95% bootstrap intervals.

Layer	Does	Key evidence	Verdict
LB memo	exact cache of mode extraction	0 harmed in 77 pairs; time ratio 0.995 [0.992, 1.000]	go, on
LB skip	skip optional steps	saves 0.2–0.7%; skips nothing on test	no-go
V1	flags likely-wrong answers	AUROC 0.847; detection 0.00–0.08 at $\alpha = 0.05$	no-ship
L2b	calibrates P(top)	raw ≥ 0.999 on 96% of rows; ΔBrier -0.032 $[-0.073, +0.007]$	no-go
L2c	rescales intervals	over-covers (0.965, 0.982 vs 0.895)	no-go
L2d	global constants	every fold picks today's values	no-go
H0	high-κ scan	identical output on 80/80, 40/40, 22/22	no-go
L0	seeds for the fit	true poles as seeds: no gain	closed
C	reruns with other settings	oracle 8/12; with V1 trigger 0/12	not built

pipeline the scan gave identical labels and factors on 80/80 test pairs, 40/40 nulls and 22/22 named cases (one timeout difference among 30 scattered runs) and only added time: the cases it can help with end as ID-KER or UNDECIDED anyway. *No-go.* L0, seeds for the exponential fit, was closed earlier: even the true poles as starting values do not improve the per-bin step, because the estimator is already efficient and its misses are limited by information, not by initialisation.

7.4 The controller (C)

A controller would read a diagnosis and rerun the pipeline with one of thirteen alternative settings. We measured the ceiling first. On the final pipeline we took every off failure that did not time out in corpus V and its matched control (6 false-confident and 2 abstentions) plus four timeouts, and ran fifteen single actions from today's defaults on each (12 cases). The *oracle* ceiling, any action rescues the case, is $8/12 = 67\%$; `lower_degree` rescues 8, `varpro` 3, and the rest at most 2. With V1 as the trigger the ceiling is 0/12: V1 flags none of the failures, and it raises 3 false alarms on 86 cases the pipeline gets right. The action space has room; *a trigger with resolution on false-confident runs is the open problem.* The controller was not built. An earlier controller version passed forking-path null tests at every depth but had an oracle ceiling of 1/13.

8 Lessons

Average separation is not tail detection V1 has a good AUROC and almost no detection at a 5% false-alarm rate. A gate at a fixed operating point measures something different from a ranking metric, and the second can look good while the first fails.

A calibrator needs an input with resolution Both probability calibrators see a saturated input. A map from a saturated number can only learn a base rate, and base rates differ by family, so the gain appears in the low-base-rate family and the damage in the others.

An oracle ceiling must be measured before building a controller Whether the action space has room (it does: 67%) is a different question from whether anything points at the cases that need it (nothing does: 0%). Measuring the ceiling cost 180 runs; building the controller would have cost far more.

Baselines move A speed-up of 7% became 0.5% when the pipeline improved; an interval calibration that raised coverage from 0.76 to 0.92 over-covered once the defect it repaired was fixed. Every gate has to be re-run on the final pipeline, which is why the contract keeps failed layers in the repository.

Draw variance dominates small corpora One base draw supplied 21 of 28 injected rows and inflated the headline AUROC from 0.85 to 0.96; cluster bootstrap and fresh draws are not optional.

Crashes and timeouts are outside any output-based detector A detector that reads the output cannot see runs that produce none; 48 of 140 injected runs were of this kind.

9 Ideas not yet tried

These are not results. They are directions the evidence suggests, or that the author proposed during the project.

1. **An input-only validator** that runs before the pipeline and predicts crashes, timeouts and out-of-class data (aliasing, coloured noise) from the data alone. It would cover the failures V1 cannot see.
2. **A trigger with resolution on false-confident runs.** The controller's ceiling is 67%; any trigger that flags even a third of the false-confident answers would change the verdict. Candidates are residual tests on raw data that are sensitive in the tail, rather than a learned score.
3. **Encoding and grouping.** Prepare the data with a nonlinear embedding (t-SNE or an autoencoder) so that the modes of different factors separate naturally, then group with a mixture model; and ask whether encoding plus grouping drives the interaction term to zero. The method should still work without this step.
4. **Regularisation by the number of factors rather than by coefficient size.** An ℓ_1 penalty on the weights of the expansion instead of ℓ_2 favours few components; whether it removes the interaction term is open.
5. **Port the layers to the current pipeline.** The layers were built on an earlier version of the pipeline (Section 10). Porting their hooks to the current one, which has a better refinement stage and fewer false-confident answers, would test whether the negative verdicts survive.

10 Reproducibility and limits

This branch holds the layers, their tests and the experiment scripts. The pipeline in `src/` is the snapshot the layers were developed and tested on (the main pipeline at commit `eaa8f9c` plus the layers). The current pipeline on branch `main` has later changes in `loop.py` and the structured fit that touch the same lines as the layer hooks, so the port in item 5 of Section 9

is real work and is not done here. All numbers in this paper are from stored reports in `benchmarks/ml_layers/results/`. The off-snapshot tests compare against stored outputs whose last digits depend on the numerical library; the stored outputs were regenerated on Linux (NumPy/SciPy as pinned in `pyproject.toml`) and the test suite passes there. Corpora are simulated; no real data are used, and the findings are about this pipeline and this simulator prior.

11 Conclusion

Under a contract that forbids a learned layer from deciding, the pipeline gains almost nothing from learning on top of it: one exact speed-up, one failure detector that ranks well and flags poorly, and a negative result for every calibration layer. The most valuable outcome is the measurement of where the remaining error lives: false-confident answers that no output-based score points at, and a rerun space that could rescue two thirds of them if a trigger existed. Finding that trigger, not adding a layer, is the next step.

References

[1] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations*, 2024. arXiv:2208.02814.

[2] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. Predictive inference with the jackknife+. *The Annals of Statistics*, 49(1):486–507, 2021.

[3] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[4] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.